UNIVERSIDAD POLITÉCNICA SALESIANA ECUADOR

POSGRADOS

Maestría en

SOFTWARE

Asignatura:

Patrones de Diseño de Software

Actividad #: 03

Patrones de comportamiento

Docente:
Mauricio Ortiz Ochoa

Elaborado por (Grupo 02):
Espinoza Adrian
Jara Pablo
Lara Miguel
Torres Carina
Uyaguari Sebastián

Fecha de entrega:
jueves, 29 de octubre de 2026

Repositorio de código:
https://github.com/carytnas/PDS_Actividad03_Grupo02

Actividad de aprendizaje #: 03

CC BY NC ND

Patrones de comportamiento

Introducción

Se eligieron dos patrones de comportamiento del catálogo propuesto: Mediator, aplicado a una torre de control que coordina el aterrizaje de varias aeronaves en una sola pista, y Command, aplicado a un control remoto que enciende y apaga una luz. Ambos están implementados en Java en el repositorio del grupo.

Patrón 01: Mediator – Torre de control de tráfico aéreo

Escenario / Motivación

Varias aeronaves quieren aterrizar en la misma pista. Solo una puede usarla a la vez; las demás esperan en el orden en que pidieron turno y se les avisa cuando la pista queda libre. Cada aeronave sabe reportar su posición, pedir autorización y aterrizar. Decidir quién usa la pista y llevar la cola no le corresponde.

Problema de diseño identificado

Sin un coordinador, cada aeronave tendría que comunicarse con todas las demás (Figura 1). Con n aeronaves hay n·(n−1) dependencias, las reglas de la pista quedan repartidas entre todas las clases y cada avión nuevo obliga a modificar los existentes. Lo que hay que ordenar es la interacción: quién usa la pista, en qué orden y quién avisa a quién.

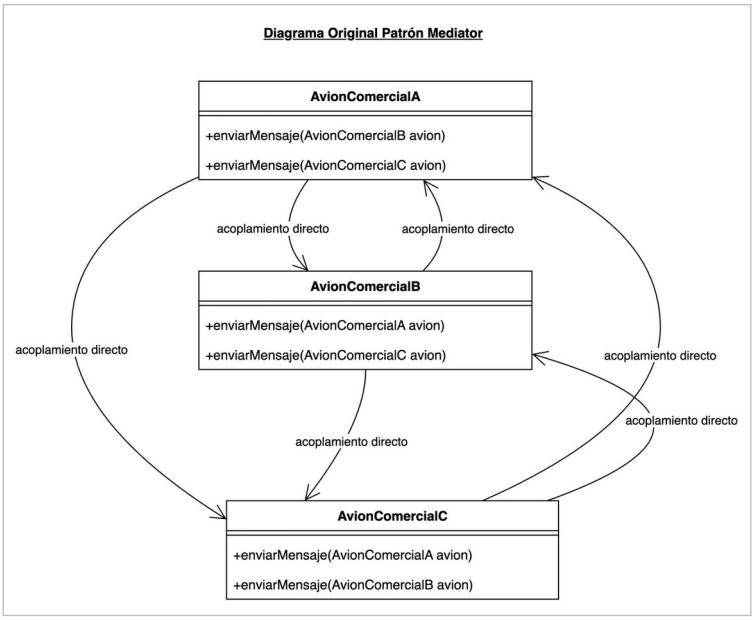


Figura 1: *Diagrama original, sin Mediator*

Justificación del patrón

Mediator encapsula en un objeto la forma en que interactúa un grupo de objetos, de modo que estos no se refieren entre sí (Gamma et al., 1994). Es lo que pide el escenario: la torre guarda el estado de la pista y la cola, y las aeronaves solo conocen la interfaz ITorreControl.

Alternativas descartadas:

- **Comunicación directa.** Es la Figura 1: el acoplamiento crece con cada aeronave que se agrega.
- **Observer.** Sirve para avisar «pista libre», pero no decide a quién le toca. Todas las aeronaves en espera recibirían el aviso a la vez y cada una tendría que resolver el turno por su cuenta.
- **Chain of Responsibility.** Pasa la solicitud por una cadena hasta que alguien la atiende. Aquí siempre la atiende la torre, y la torre necesita ver la pista y la cola completas.
- **Facade.** Simplifica el acceso a un subsistema en un solo sentido. La torre, en cambio, también avisa a las aeronaves (recibirAutorizacion, recibirEspera), y esa comunicación de ida y vuelta es la que resuelve Mediator.

Participantes del patrón

- **ITorreControl (Mediator):** define registrarAeronave(), solicitarAterrizaje() y liberarPista().
- **TorreControlConcreta (ConcreteMediator):** guarda las aeronaves registradas, la cola de espera y la aeronave en pista. Autoriza o encola cada solicitud y, cuando se libera la pista, autoriza a la siguiente.
- **Aeronave (Colleague):** clase abstracta. Se registra en la torre al crearse, pide autorización y recibe las respuestas de la torre.
- **AvionComercial (ConcreteColleague):** aterriza solo si tiene autorización y después libera la pista.
- **MainMediatorTorreControl (Client):** arma la simulación con tres vuelos.

En la ejecución (Figura 3), VUELO-LA402 encuentra la pista libre y recibe autorización; VUELO-IB612 y VUELO-AA990 quedan en cola en las posiciones 1 y 2. Cada vez que un vuelo aterriza y libera la pista, la torre autoriza al siguiente. Ningún avión llama a otro.

Diagrama de clases UML

Notación UML según Fowler (2004). Las aeronaves apuntan a ITorreControl (0..* a 1) y TorreControlConcreta guarda listaAeronaves y colaEspera (1 a 0..*). MainMediatorTorreControl crea la torre y los aviones (dependencias «create»). No hay asociaciones entre aeronaves.

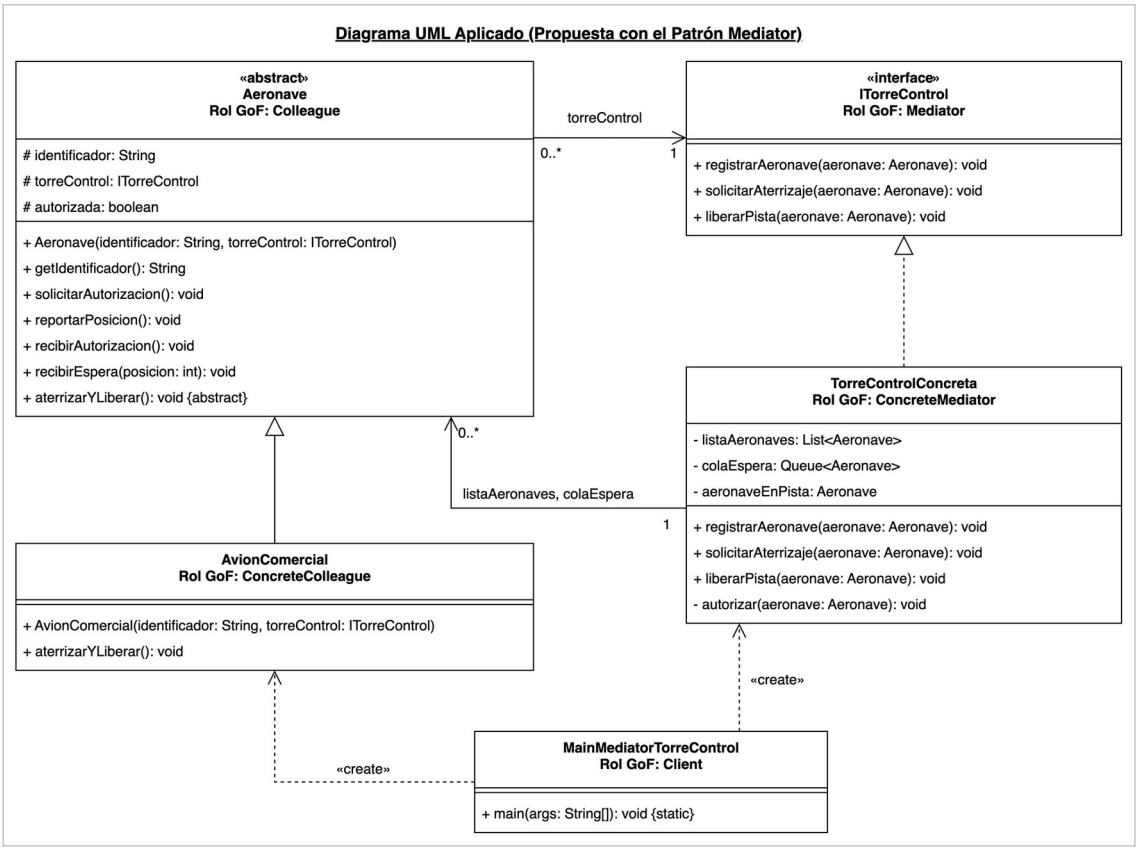


Figura 2: Diagrama UML con Mediator

Código fuente

Repositorio / enlace: https://github.com/carytnas/PDS_Actividad03_Grupo02/tree/main/codigo/ejercicios/src/mediator

La clase central es TorreControlConcreta:

src/mediator/TorreControlConcreta.java

```
package mediator;


import java.util.ArrayList;
import java.util.LinkedList;
import java.util.List;
import java.util.Queue;


public class TorreControlConcreta implements ITorreControl {


  private List<Aeronave> listaAeronaves = new ArrayList<Aeronave>();
  private Queue<Aeronave> colaEspera = new LinkedList<Aeronave>();
  private Aeronave aeronaveEnPista = null;


  @Override
  public void registrarAeronave(Aeronave aeronave) {
    listaAeronaves.add(aeronave);
  }


  @Override
  public void solicitarAterrizaje(Aeronave aeronave) {
    System.out.println("\n[TORRE] Solicitud recibida de la aeronave: " +
aeronave.getIdentificador());


    if (!listaAeronaves.contains(aeronave)) {
```

```
      System.out.println("[TORRE AVISO] La aeronave " + aeronave.getIdentificador() + " no está
registrada.");
      return;
    }
    if (aeronave == aeronaveEnPista || colaEspera.contains(aeronave)) {
      System.out.println("[TORRE AVISO] La aeronave " + aeronave.getIdentificador()
          + " ya tiene la pista asignada o está en la cola de espera.");
      return;
    }

    if (aeronaveEnPista == null) {
      autorizar(aeronave);
    } else {
      colaEspera.add(aeronave);
      System.out.println("[TORRE] Pista ocupada actualmente por [" +
aeronaveEnPista.getIdentificador() + "]. "
          + aeronave.getIdentificador() + ", mantenga el patrón de espera.");
      aeronave.recibirEspera(colaEspera.size());
    }
  }

  @Override
  public void liberarPista(Aeronave aeronave) {
    if (aeronaveEnPista == aeronave) {
      System.out.println("[TORRE] La aeronave " + aeronave.getIdentificador() + " ha liberado la
pista.");
      aeronaveEnPista = null;

      Aeronave siguiente = colaEspera.poll();
      if (siguiente != null) {
        autorizar(siguiente);
      }
    } else {
      System.out.println("[TORRE AVISO] La aeronave " + aeronave.getIdentificador()
          + " intentó liberar la pista pero no estaba asignada a ella.");
    }
  }

  private void autorizar(Aeronave aeronave) {
    aeronaveEnPista = aeronave;
    System.out.println("[TORRE] Autorizado el aterrizaje para: " + aeronave.getIdentificador() + ".
Pista ocupada.");
    aeronave.recibirAutorizacion();
  }
}
```

```
=== SIMULACIÓN DE TRÁFICO AÉREO (PATRÓN MEDIATOR) ===
[VUELO-LA402] Reportando posición actual en el espacio aéreo.
[VUELO-IB612] Reportando posición actual en el espacio aéreo.
[VUELO-AA990] Reportando posición actual en el espacio aéreo.

[TORRE] Solicitud recibida de la aeronave: VUELO-LA402
[TORRE] Autorizado el aterrizaje para: VUELO-LA402. Pista ocupada.
[VUELO-LA402] Autorización recibida. Procediendo al aterrizaje.

[TORRE] Solicitud recibida de la aeronave: VUELO-IB612
[TORRE] Pista ocupada actualmente por [VUELO-LA402]. VUELO-IB612, mantenga el patrón de espera.
[VUELO-IB612] En patrón de espera (posición 1 en la cola).

[TORRE] Solicitud recibida de la aeronave: VUELO-AA990
[TORRE] Pista ocupada actualmente por [VUELO-LA402]. VUELO-AA990, mantenga el patrón de espera.
[VUELO-AA990] En patrón de espera (posición 2 en la cola).

[VUELO-LA402] Realizando maniobra de aterrizaje...
[TORRE] La aeronave VUELO-LA402 ha liberado la pista.
[TORRE] Autorizado el aterrizaje para: VUELO-IB612. Pista ocupada.
[VUELO-IB612] Autorización recibida. Procediendo al aterrizaje.

[VUELO-IB612] Realizando maniobra de aterrizaje...
[TORRE] La aeronave VUELO-IB612 ha liberado la pista.
[TORRE] Autorizado el aterrizaje para: VUELO-AA990. Pista ocupada.
[VUELO-AA990] Autorización recibida. Procediendo al aterrizaje.

[VUELO-AA990] Realizando maniobra de aterrizaje...
[TORRE] La aeronave VUELO-AA990 ha liberado la pista.
```

Figura 3: *Salida de MainMediatorTorreControl*

Validación de la solución

- **Qué resolvió:** las reglas de la pista quedaron en un solo lugar y los aviones dejaron de conocerse entre sí.
- **Qué mejoró:** hay una sola dependencia (ITorreControl) en lugar de n·(n−1). Agregar un avión no afecta a los demás, y la regla de turnos se puede cambiar con otra implementación de ITorreControl.
- **Costo:** la torre concentra toda la lógica y puede crecer demasiado. El flujo avión → torre → avión es más difícil de seguir.
- **Cuándo no usarlo:** si interactúan pocos objetos de forma simple, el mediador solo agrega una capa (Refactoring.Guru, s. f.-b).

Uso crítico de inteligencia artificial

- **Prompt principal:**

"Diseña una solución orientada a objetos en Java aplicando el patrón Mediator para coordinar el tráfico y las autorizaciones de aterrizaje entre múltiples aeronaves y una torre de control centralizada, evitando la comunicación directa entre los aviones."

- **Recomendación aceptada y justificación:**

Centralizar el estado de la pista y la validación de turnos en TorreControlConcreta. Se aceptó porque así las aeronaves no toman decisiones de tráfico y cada clase se limita a su rol en el patrón.

- **Recomendación modificada o rechazada y justificación:**

Que cada aeronave tuviera una lista de las demás para avisarse entre sí ante emergencias en pista. Se rechazó porque los colegas no deben conocerse: una emergencia se reporta a la torre.

Patrón 02: Command – Control remoto de luces

Escenario / Motivación

Un control remoto enciende y apaga una luz. El control decide cuándo actuar (al presionar el botón) y la luz sabe cómo hacerlo. El mismo botón debe poder cambiar de acción mientras el programa está en ejecución.

Problema de diseño identificado

Si el control llama directamente a la luz con presionarBotonEncender() y presionarBotonApagar() (Figura 4), queda atado a ese dispositivo y a esas dos acciones. Cada acción o dispositivo nuevo obliga a modificar el control, y no hay forma de reasignar un botón en ejecución. La solicitud tiene que convertirse en un objeto que el control pueda ejecutar sin saber qué hay detrás.

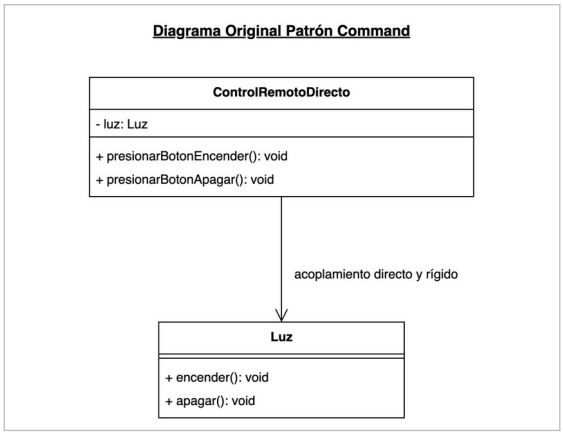


Figura 4: *Diagrama original, sin Command*

Justificación del patrón

Command encapsula una solicitud como objeto; así se puede parametrizar a quien la invoca, encolarla o registrarla (Gamma et al., 1994). El control solo llama a ejecutar(), y lo que se ejecuta depende del comando que tenga asignado.

Alternativas descartadas:

- **Condicionales en el control.** Un presionarBoton(accion) con un switch. Hay que editarlo con cada acción nueva y sigue dependiendo de cada dispositivo.
- **Strategy.** La estructura es casi igual, pero Strategy cambia cómo un objeto hace su propio trabajo. Aquí lo que cambia es qué orden se envía y a quién, y Command guarda al receptor dentro del objeto.

- **Observer.** El botón avisaría «me presionaron» y la luz decidiría qué hacer. La decisión vuelve a quedar en condicionales, ahora dentro de la luz, y cambiar la acción del botón significa cambiar suscripciones.
- **Mediator.** No hay varios objetos que coordinar, solo uno que pide y otro que ejecuta.

Participantes del patrón

- **IComando (Command):** declara ejecutar().
- **EncenderLuzCommand y ApagarLuzCommand (ConcreteCommand):** reciben la Luz en el constructor y en ejecutar() llaman a encender() o apagar().
- **Luz (Receiver):** hace el trabajo real.
- **ControlRemoto (Invoker):** guarda un IComando con setComando() y lo ejecuta en presionarBoton(). Si no tiene comando asignado, avisa y no hace nada.
- **MainCommandControlLuces (Client):** crea la luz, los comandos y el control, y los conecta.

En la ejecución (Figura 6), el cliente asigna EncenderLuzCommand, presiona el botón y la luz se enciende; después asigna ApagarLuzCommand al mismo botón y la luz se apaga. ControlRemoto nunca ve a la Luz.

Diagrama de clases UML

ControlRemoto tiene una agregación hacia IComando (0..1). Los dos comandos implementan IComando y delegan en Luz. MainCommandControlLuces crea los objetos (dependencias «create»).

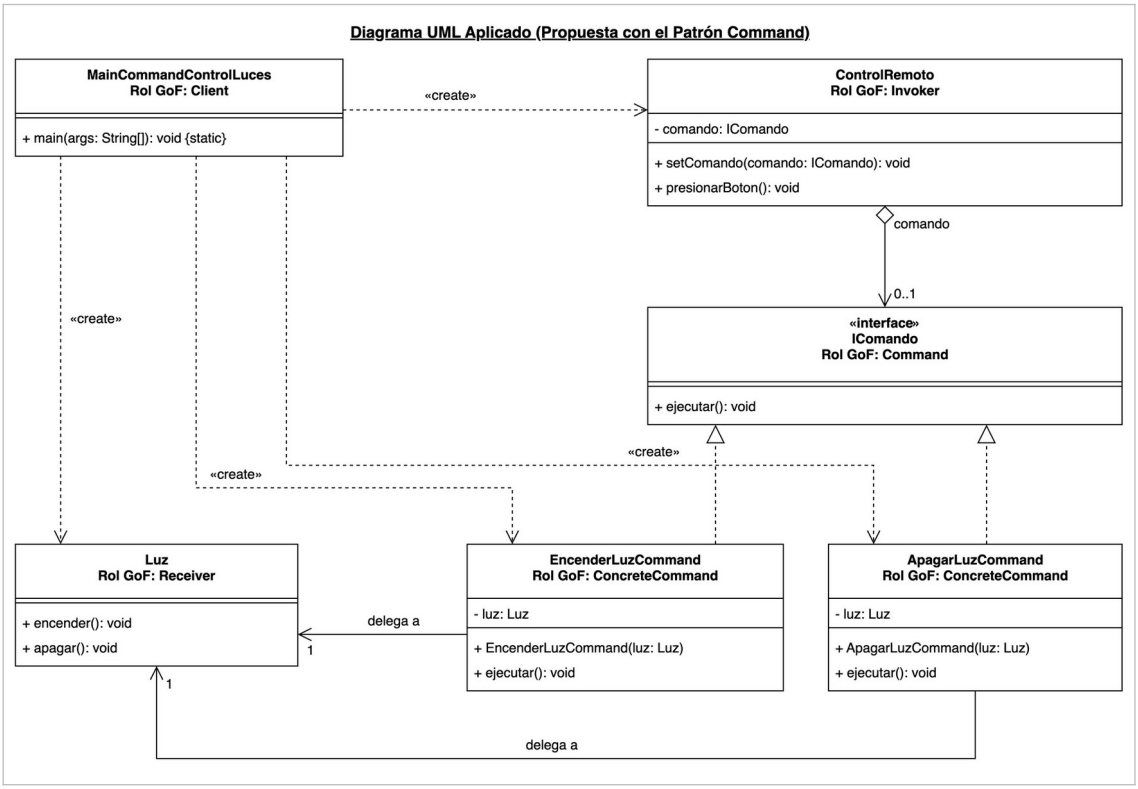


Figura 5: *Diagrama UML con Command*

Código fuente

Repositorio / enlace: https://github.com/carytnas/PDS_Actividad03_Grupo02/tree/main/codigo/ejercicios/src/command

Se muestran el invocador y uno de los comandos:

src/command/ControlRemoto.java

```
package command;


public class ControlRemoto {
  private IComando comando;


  public void setComando(IComando comando) {
    this.comando = comando;
  }


  public void presionarBoton() {
    if (comando == null) {
      System.out.println("No hay ningún comando asignado en el control remoto.");
      return;
    }
    comando.ejecutar();
  }
}
```

src/command/EncenderLuzCommand.java

```
package command;


public class EncenderLuzCommand implements IComando {
  private Luz luz;


  public EncenderLuzCommand(Luz luz) {
    this.luz = luz;
  }


  @Override
  public void ejecutar() {
    System.out.println("[Comando] Ejecutando orden: Encender luz.");
    luz.encender();
  }
}
```

```
=== ACCIÓN 1: ENCENDER LA LUZ ===
[Comando] Ejecutando orden: Encender luz.
-> [Luz] El dispositivo está ENCENDIDO.

=== ACCIÓN 2: APAGAR LA LUZ ===
[Comando] Ejecutando orden: Apagar luz.
-> [Luz] El dispositivo está APAGADO.
```

Figura 6: *Salida de MainCommandControlLuces*

Validación de la solución

- **Qué resolvió:** el control ya no depende de la luz y un botón puede cambiar de acción en ejecución.
- **Qué mejoró:** una acción nueva es una clase nueva; ControlRemoto no se modifica y no tiene condicionales. Además, deja la puerta abierta a un historial o a deshacer, que no están implementados.

- **Costo:** una clase por acción y una llamada intermedia. Deshacer exigiría guardar el estado anterior de la luz.
- **Cuándo no usarlo:** si hay una sola operación fija que nunca se va a reasignar ni registrar (Refactoring.Guru, s. f.-a).

Uso crítico de inteligencia artificial

- **Prompt principal:**

"Diseña una solución orientada a objetos en Java aplicando el patrón Command para modelar un sistema de control remoto que accione dispositivos inteligentes (como luces), desacoplando los botones de la lógica interna de los aparatos mediante la encapsulación de solicitudes en objetos ejecutables."

- **Recomendación aceptada y justificación:**

Una interfaz IComando con un único método ejecutar(), y que cada comando concreto guarde una referencia a Luz. Se aceptó porque ControlRemoto puede ejecutar cualquier comando sin saber qué dispositivo ni qué acción hay detrás.

- **Recomendación modificada o rechazada y justificación:**

Poner la lógica directamente en ejecutar(), sin un receptor. Se rechazó porque el comando solo debe delegar: la lógica del dispositivo va en Luz. De lo contrario, cada comando acumula lógica de dominio.

Síntesis comparativa: Mediator y Command

Criterio	Mediator	Command
Pregunta que responde	¿Cómo coordinamos varios objetos?	¿Cómo representamos una solicitud?
Centro del patrón	Reglas de interacción	Objeto Command
Dependencia reducida	Colega ↔ colega	Invoker ↔ Receiver
Ejemplo implementado	Torre de control	Control remoto
Riesgo principal	Mediator monolítico	Proliferación de clases

Para elegir entre ambos, primero hay que ver si el problema es coordinar varios objetos o representar una solicitud.

Conclusiones

La elección de cada patrón dependió de ubicar bien el problema. En la torre de control el problema era de coordinación: varias aeronaves compiten por una pista y, sin un punto central, cada una tendría que conocer a las demás. Con Mediator, TorreControlConcreta lleva el estado de la pista y la cola, y cada aeronave solo reporta, pide turno y aterriza. En el control remoto el problema era cómo representar la

orden. Con Command, ControlRemoto llama a ejecutar() sin saber que del otro lado hay una luz, y el mismo botón cambia de acción en ejecución.

Ninguno de los dos patrones es gratuito. La torre concentra toda la lógica y puede volverse difícil de mantener si se le siguen agregando reglas; Command suma una clase por cada acción y una llamada intermedia. En estos dos casos el costo se justifica, pero en un problema más simple no se justificaría.

La IA sirvió para obtener una primera propuesta de estructura, pero cada sugerencia se revisó contra los roles GoF. Se aceptaron las que mantenían separadas las responsabilidades y se rechazaron las que volvían a acoplar a las aeronaves entre sí o ponían la lógica de la luz dentro de los comandos.

Referencias bibliográficas

Fowler, M. (2004). *UML distilled: A brief guide to the standard object modeling language* (3.ª ed.). Addison-Wesley.

Gamma, E., Helm, R., Johnson, R., y Vlissides, J. (1994). *Design patterns: Elements of reusable object-oriented software*. Addison-Wesley.

Oracle. (s. f.). *Java documentation*. https://docs.oracle.com/en/java/

Refactoring.Guru. (s. f.-a). *Command*. https://refactoring.guru/es/design-patterns/command

Refactoring.Guru. (s. f.-b). *Mediator*. https://refactoring.guru/es/design-patterns/mediator

Documentos

- Unidad_03_01_PDS.pdf
- Unidad_03_02_PDS.pdf